



Framework Overview

Introduction

The Odoo Javascript framework is a set of features/building blocks provided by the `web/` addon to help build odoo applications running in the browser. At the same time, the Odoo Javascript framework is a single page application, usually known as the *web client* (available at the url `/web`).

The web client started as an application made with a custom class and widget system, but it is now transitioning to using native javascript classes instead, and Owl as a component system. This explains why both systems are currently in use in the codebase.

From a high-level perspective, the web client is a single-page application: it does not need to request a full page from the server each time the user performs an action. Instead, it only requests what it needs and then replaces/updates the current screen accordingly. Also, it manages the url to keep it in sync with the current state.

The javascript framework (all or some parts) is also used in other situations, such as the Odoo website or the point of sale. This reference is mostly focused on the web client.

Note

It is common in the Odoo ecosystem to see the words *frontend* and *backend* as synonyms for the odoo website (public) and the web client, respectively. This terminology is not to be confused with the more common use of browser-code (frontend) and server (backend).

Note

In this documentation, the word *component* always refers to new Owl components, and *widget* refers to old Odoo widgets.

Note

All new development should be done in Owl, if possible!

codebase. Here is a list of the most important folders:

- `core/` most of the low level features
- `fields/` all field components
- `views/` all javascript views components (`form` , `list` , ...)
- `search/` control panel, search bar, search panel, ...
- `webclient/` the web client specific code: navbar, user menu, action service, ...

The `web/static/src` is the root folder. Everything inside can simply be imported by using the `@web` prefix. For example, here is how one can import the `memoize` function located in `web/static/src/core/utils/functions` :

```
import { memoize } from "@web/core/utils/functions";
```

WebClient Architecture

As mentioned above, the web client is an owl application. Here is a slightly simplified version of its template:

```
<t t-name="web.WebClient">
  <body class="o_web_client">
    <NavBar/>
    <ActionContainer/>
    <MainComponentsContainer/>
  </body>
</t>
```

As we can see, it basically is a wrapper for a navbar, the current action and some additional components. The `ActionContainer` is a higher order component that will display the current action controller (so, a client action, or a specific view in the case of actions of type `act_window`). Managing actions is a huge part of its work: the action service keeps in memory a stack of all active actions (represented in the breadcrumbs), and coordinates each change.

Another interesting thing to note is the `MainComponentsContainer` : it is simply a component that displays all components registered in the `main_components` registry. This is how other parts of the system can extend the web client.

access it using `this.env`). Here is a description of what Odoo adds to the shared `env` object:

Key	Value
<code>qweb</code>	required by Owl (contains all templates)
<code>bus</code>	main bus , used to coordinate some generic events
<code>services</code>	all deployed services (should usually be accessed with the <code>useService</code> hook)
<code>debug</code>	string. If non empty, the web client is in debug mode
<code>_t</code>	translation function
<code>isSmall</code>	boolean. If true, the web client is currently in mobile mode (screen width ≤ 767 px)

So, for example, to translate a string in a component (note: templates are automatically translated, so no specific action is required in that case), one can do this:

```
const someString = this.env._t('some text');
```

Note

Having a reference to the environment is quite powerful, because it provides access to all services. This is useful in many cases: for example, user menu items are mostly defined as a string, and a function taking the `env` as unique argument. This is enough to express all user menu needs.

Building Blocks

Most of the web client is built with a few types of abstractions: registries, services, components and hooks.

Registries

(or widgets) that can be used in views.

```
import { Component } from "@odoo/owl";
import { registry } from "../core/registry";

class MyFieldChar extends Component {
  // some code
}

registry.category("fields").add("my_field_char", MyFieldChar);
```

Note that we import the main registry from `@web/core/registry` then open the sub registry `fields`.

Services

Services are long lived pieces of code that provide a feature. They may be imported by components (with `useService`) or by other services. Also, they can declare a set of dependencies. In that sense, services are basically a DI (dependency injection) system. For example, the `notification` service provides a way to display a notification, or the `rpc` service is the proper way to perform a request to the Odoo server.

The following example registers a simple service that displays a notification every 5 second:

```
import { registry } from "../core/registry";

const serviceRegistry = registry.category("services");

const myService = {
  dependencies: ["notification"],
  start(env, { notification }) {
    let counter = 1;
    setInterval(() => {
      notification.add(`Tick Tock ${counter++}`);
    }, 5000);
  }
};

serviceRegistry.add("myService", myService);
```

Hooks [↗](#) are a way to factorize code, even if it depends on lifecycle. This is a composable/functional way to inject a feature in a component. They can be seen as a kind of mixin.

```
function useCurrentTime() {  
  const state = useState({ now: new Date() });  
  const update = () => state.now = new Date();  
  let timer;  
  onWillStart(() => timer = setInterval(update, 1000));  
  onWillUnmount(() => clearInterval(timer));  
  return state;  
}
```

Context

An important concept in the Odoo javascript is the *context*: it provides a way for code to give more context to a function call or a rpc, so other parts of the system can properly react to that information. In some way, it is like a bag of information that is propagated everywhere. It is useful in some situations, such as letting the Odoo server know that a model rpc comes from a specific form view, or activating/disabling some features in a component.

There are two different contexts in the Odoo web client: the *user context* and the *action context* (so, we should be careful when using the word *context*: it could mean a different thing depending on the situation).

Note

The `context` object may be useful in many cases, but one should be careful not to overuse it! Many problems can be solved in a standard way without modifying the context.

User Context

The *user context* is a small object containing various informations related to the current user. It is available through the `user` service:

```
        console.log(user.context);  
    }  
}
```

It contains the following information:

Name	Type	Description
allowed_company_ids	number[]	the list of active company ids for the user
lang	string	the user language code (such as "en_us")
tz	string	the user current timezone (for example "Europe/Brussels")

In practice, the `orm` service automatically adds the user context to each of its requests. This is why it is usually not necessary to import it directly in most cases.

Note

The first element of the `allowed_company_ids` is the main company of the user.

Action Context

The `ir.actions.act_window` and `ir.actions.client` support an optional `context` field. This field is a `char` that represents an object. Whenever the corresponding action is loaded in the web client, this context field will be evaluated as an object and given to the component that corresponds to the action.

```
<field name="context">{'search_default_customer': 1}</field>
```

It can be used in many different ways. For example, the views add the action context to every requests made to the server. Another important use is to activate some search filter by default (see example above).

Sometimes, when we execute new actions manually (so, programmatically, in javascript), it is useful to be able to extend the action context. This can be done with the `additional_context`

```
let actionService = useService("action");

// in some event handler
actionService.doAction("addon_name.something", {
  additional_context:{
    default_period_id: defaultPeriodId
  }
});
```

In this example, the action with `xml_id` `addon_name.something` will be loaded, and its context will be extended with the `default_period_id` value. This is a very important usecase that lets developers combine actions together by providing some information to the next action.

Python Interpreter

The Odoo framework features a built-in small python interpreter. Its purpose is to evaluate small python expressions. This is important, because views in Odoo have modifiers written in python, but they need to be evaluated by the browser.

Example:

```
import { evaluateExpr } from "@web/core/py_js/py";

evaluateExpr("1 + 2*{'a': 1}.get('b', 54) + v", { v: 33 }); // returns 142
```

The `py` javascript code exports 5 functions:

`tokenize(expr)`

Arguments:

`expr ( string() )` – the expression to tokenize

Returns:

`Token[]` a list of token

`parse(tokens)`

Arguments:

`tokens ( Token[]() )` – a list of tokens

Returns:

Arguments:

expr (string()) – a string representing a valid python expression

Returns:

AST an abstract syntax tree structure representing the expression

evaluate(ast[, context])**Arguments:**

ast (AST()) – a AST structure that represents an expression

context (Object()) – an object that provides an additional evaluation context

Returns:

any the resulting value of the expression, with respect to the context

evaluateExpr(expr[, context])**Arguments:**

expr (string()) – a string representing a valid python expression

context (Object()) – an object that provides an additional evaluation context

Returns:

any the resulting value of the expression, with respect to the context

Domains

Broadly speaking, domains in Odoo represent a set of records that matches some specified conditions. In javascript, they are usually represented either as a list of conditions (or of operators: | , & or ! in prefix notation), or as string expressions. They don't have to be normalized (the & operator is implied if necessary). For example:

```
// list of conditions
[
  ["a", "=", 3]]
[["a", "=", 1], ["b", "=", 2], ["c", "=", 3]]
["&", "&", ["a", "=", 1], ["b", "=", 2], ["c", "=", 3]]
["&", "!", ["a", "=", 1], "|", ["a", "=", 2], ["a", "=", 3]]

// string expressions
"(['some_file', '>', a])"
```


String expressions are more powerful than list expressions: they can contain python expressions and unevaluated values, that depends on some evaluation context. However, manipulating string expressions is more difficult.

Since domains are quite important in the web client, Odoo provides a `Domain` class:

```
new Domain(["a", "=", 3]).contains({ a: 3 }) // true

const domain = new Domain("&", "&", ["a", "=", 1], ["b", "=", 2], ["c", "=", 3]);
domain.contains({ a: 1, b: 2, c: 3 }); // true
domain.contains({ a: -1, b: 2, c: 3 }); // false

// next expression returns ["|", ("a", "=", 1), ("b", "<=", 3)]
Domain.or([["a", "=", 1], ["('b', '<=', 3)"]]).toString();
```

Here is the `Domain` class description:

`class Domain([descr])`

Arguments:

descr (`string` | `any[]` | `Domain()`) – a domain description

`Domain.contains (record)`

Arguments:

record (`Object()`) – a record object

Returns:

boolean

Returns true if the record matches all the condition specified by the domain

`Domain.toString ()`

Returns:

string

Returns a string description for the domain

Returns:

any[]

Returns a list description for the domain. Note that this method takes an optional `context` object that will be used to replace all free variables.

```
new Domain(`(['a', '>', b])`).toList({ b:3 }); // [['a', '>', 3]]
```

The `Domain` class also provides 4 useful static methods to combine domains:

```
// ["&", ("a", "=", 1), ("uid", "<=", uid)]
Domain.and([["a", "=", 1], ["('uid', '<=', uid)"]]).toString();

// ["|", ("a", "=", 1), ("uid", "<=", uid)]
Domain.or([["a", "=", 1], ["('uid', '<=', uid)"]]).toString();

// ["!", ("a", "=", 1)]
Domain.not([["a", "=", 1]]).toString();

// ["&", ("a", "=", 1), ("uid", "<=", uid)]
Domain.combine([["a", "=", 1], ["('uid', '<=', uid)"]], "AND").toString();
```

static Domain.and (domains)**Parameters:**

domains (*string[]* | *any[][]* | *Domain[]*) – a list of domain representations

Returns:

Domain

Returns a domain representing the intersection of all domains.

static Domain.or (domains)**Parameters:**

domains (*string[]* | *any[][]* | *Domain[]*) – a list of domain representations

Returns:

Domain

Returns a domain representing the union of all domains.

Returns:

Domain

Returns a domain representing the negation of the domain argument

`static Domain.combine(domains, operator)`**Parameters:**

domains (`string[]` / `any[][]` / `Domain[]`) – a list of domain representations

operator (`'AND'` or `'OR'`) – an operator

Returns:

Domain

Returns a domain representing either the intersection or the union of all the domains, depending on the value of the operator argument.

Bus

The web client **environment** object contains an event bus, named `bus`. Its purpose is to allow various parts of the system to properly coordinate themselves, without coupling them. The `env.bus` is an owl **EventBus**[↗](#), that should be used for global events of interest.

```
// for example, in some service code:
env.bus.on("WEB_CLIENT_READY", null, doSomething);
```

Here is a list of the events that can be triggered on this bus:

Message	Payload	Trigger
ACTION_MANAGER:UI-UPDATED	a mode indicating what part of the ui has been updated ('current', 'new' or 'fullscreen')	the rendering of the action requested to the action manager is done
ACTION_MANAGER:UPDATE	next rendering info	the action manager has finished computing the next interface

		app has changed
ROUTE_CHANGE	none	the url hash was changed
RPC:REQUEST	rpc id	a rpc request has just started
RPC:RESPONSE	rpc id	a rpc request is completed
WEB_CLIENT_READY	none	the web client has been mounted
FOCUS-VIEW	none	the main view should focus itself
CLEAR-CACHES	none	all internal caches should be cleared
CLEAR-UNCOMMITTED-CHANGES	list of functions	all views with uncommitted changes should clear them, and push a callback in the list

Browser Object

The javascript framework also provides a special object `browser` that provides access to many browser APIs, like `location`, `localStorage` or `setTimeout`. For example, here is how one could use the `browser.setTimeout` function:

```
import { browser } from "@web/core/browser/browser";

// somewhere in code
browser.setTimeout(someFunction, 1000);
```

It is mostly interesting for testing purposes: all code using the browser object can be tested easily by mocking the relevant functions for the duration of the test.

It contains the following content:

fetch	history	localStorage
location	navigator	open
random	removeEventListener	requestAnimationFrame
sessionStorage	setInterval	setTimeout
XMLHttpRequest		

Debug mode

Odoo can sometimes operate in a special mode called the `debug` mode. It is used for two main purposes:

- display additional information/fields for some particular screens,
- provide some additional tools to help developer debug the Odoo interface.

The `debug` mode is described by a string. An empty string means that the `debug` mode is not active. Otherwise, it is active. If the string contains `assets` or `tests`, then the corresponding specific sub modes are activated (see below). Both modes can be active at the same time, for example with the string `assets,tests`.

The `debug` mode current value can be read in the **environment**: `env.debug`.

Tip

To show menus, fields or view elements only in debug mode, you should target the group `base.group_no_one`:

```
<field name="fname" groups="base.group_no_one"/>
```

See also

- [Activate the debug mode](#)

Assets mode

Tests mode

There is another sub mode named `tests` : if enabled, the server injects the bundle `web.assets_tests` in the page. This bundle contains mostly test tours (tours whose purpose is to test a feature, not to show something interesting to users). The `tests` mode is then useful to be able to run these tours.

See also

- [Owl Repository](#) 

Get Help

[Contact Support](#)[Ask the Odoo Community](#)